

UNIT – 5

LEETCODE EXERCISE

1. Product Sales Analysis I

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

sale_id,year is the primary key column for this table.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key column for this table.

Problem Summary

Write a solution to report the product_name, year, and price for each sale_id in the Sales table.

SQL Schema

```
Create table Sales (sale_id int, product_id int, year int, quantity int, price int);
```

```
Create table Product (product_id int, product_name varchar(10));
```

```
Truncate table Sales;
```

```
insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');
```

```
insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');
```

```
insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');
```

```
Truncate table Product;
```

```
insert into Product (product_id, product_name) values ('100', 'Nokia');
```

```
insert into Product (product_id, product_name) values ('200', 'Apple');
```

```
insert into Product (product_id, product_name) values ('300', 'Samsung');
```

Sales

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

product_id	product_name
100	Nokia
200	Apple
300	Samsung

MySQL Solution

The screenshot shows a MySQL IDE interface. At the top, there are tabs for 'Testcase' and 'Test Result'. Below the tabs, the 'Output' window displays the result of a query in a table format:

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

Below the output window, the 'Code' window shows the MySQL query used to generate the output:

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```

2. Product Sales Analysis II

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

sale_id,year is the primary key column for this table.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key column for this table.

Problem Summary

Write an SQL query that reports the total quantity sold for every product id.

SQL Schema

Create table Sales (sale_id int, product_id int, year int, quantity int, price int);

Create table Product (product_id int, product_name varchar(10));

Truncate table Sales;

insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');

Truncate table Product;

insert into Product (product_id, product_name) values ('100', 'Nokia');

insert into Product (product_id, product_name) values ('200', 'Apple');

insert into Product (product_id, product_name) values ('300', 'Samsung');

Sales

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

product_id	product_name
100	Nokia
200	Apple
300	Samsung

MySQL Solution

```
</> Code
MySQL v Auto
1 # Write your MySQL query statement below
2 select product_id, sum(quantity) as total_quantity
3 from Sales
4 group by product_id;
```

Testcase > Test Result

Output

product_id	total_quantity
100	22
200	15

3. Product Price at a Given Date

Table: Products

Column Name	Type
product_id	int
new_price	int
change_date	date

(product_id, change_date) is the primary key (combination of columns with unique values) of this table.

Problem Summary

Each row of this table indicates that the price of some product was changed to a new price at some date.

Initially, all products have price 10.

Write a solution to find the prices of all products on the date 2019-08-16.

SQL Schema

Create table If Not Exists Products (product_id int, new_price int, change_date date)

Truncate table Products

insert into Products (product_id, new_price, change_date) values ('1', '20', '2019-08-14')

insert into Products (product_id, new_price, change_date) values ('2', '50', '2019-08-14')

insert into Products (product_id, new_price, change_date) values ('1', '30', '2019-08-15')

insert into Products (product_id, new_price, change_date) values ('1', '35', '2019-08-16')

insert into Products (product_id, new_price, change_date) values ('2', '65', '2019-08-17')

insert into Products (product_id, new_price, change_date) values ('3', '20', '2019-08-18')

Products

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

MySQL Solution

```
Code
MySQL v Auto
1 # Write your MySQL query statement below
2 WITH LatestPrice AS (
3     SELECT
4         product_id,
5         new_price,
6         ROW_NUMBER() OVER (
7             PARTITION BY product_id
8             ORDER BY change_date DESC
9         ) AS rn
10    FROM Products
11   WHERE change_date <= '2019-08-16'
12 )
13
14 SELECT
15     p.product_id,
16     COALESCE(lp.new_price, 10) AS price
17 FROM
18     (SELECT DISTINCT product_id FROM Products) p
19 LEFT JOIN LatestPrice lp
20     ON p.product_id = lp.product_id
21    AND lp.rn = 1;
```

Testcase > Test Result

Output

product_id	price
1	35
2	50
3	10

4. Immediate Food Delivery II

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int
order_date	date
customer_pref_delivery_date	date

delivery_id is the column of unique values of this table.

Problem Summary

If the customers preferred delivery date is the same as the order date, then the order is called immediate; otherwise, it is called scheduled.

The first order of a customer is the order with the earliest order date that the customer made. It is guaranteed that a customer has precisely one first order.

Write a solution to find the percentage of immediate orders in the first orders of all customers, rounded to 2 decimal places.

SQL Schema

Create table If Not Exists Delivery (delivery_id int, customer_id int, order_date date, customer_pref_delivery_date date)

Truncate table Delivery

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('1', '1', '2019-08-01', '2019-08-02')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('2', '2', '2019-08-02', '2019-08-02')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('3', '1', '2019-08-11', '2019-08-12')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('4', '3', '2019-08-24', '2019-08-24')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('5', '3', '2019-08-21', '2019-08-22')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('6', '2', '2019-08-11', '2019-08-13')

insert into Delivery (delivery_id, customer_id, order_date, customer_pref_delivery_date)
values ('7', '4', '2019-08-09', '2019-08-09')

Delivery

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	2	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-12
4	3	2019-08-24	2019-08-24
5	3	2019-08-21	2019-08-22
6	2	2019-08-11	2019-08-13
7	4	2019-08-09	2019-08-09

MySQL Solution

```
</> Code
MySQL v Auto
1 # Write your MySQL query statement below
2 WITH FirstOrders AS (
3     SELECT *,
4         ROW_NUMBER() OVER (
5             PARTITION BY customer_id
6             ORDER BY order_date
7         ) AS rn
8     FROM Delivery
9 )
10
11 SELECT
12     ROUND(
13         100.0 *
14         SUM(CASE
15             WHEN order_date = customer_pref_delivery_date
16             THEN 1
17             ELSE 0
18         END)
19         / COUNT(*),
20         2
21     ) AS immediate_percentage
22 FROM FirstOrders
23 WHERE rn = 1;
```

Testcase > Test Result

Output

immediate_percentage
50

5. Monthly Transactions I

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum
amount	int
trans_date	date

sale_id,year is the primary key column for this table.

Problem Summary

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

Return the result table in any order.

SQL Schema

Create table Sales (sale_id int, product_id int, year int, quantity int, price int);

Create table Product (product_id int, product_name varchar(10));

Truncate table Sales;

insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');

Truncate table Product;

insert into Product (product_id, product_name) values ('100', 'Nokia');

insert into Product (product_id, product_name) values ('200', 'Apple');

insert into Product (product_id, product_name) values ('300', 'Samsung');

Transactions

id	country	state	amount	trans_date
121	US	approved	1000	2018-12-18
122	US	declined	2000	2018-12-19
123	US	approved	2000	2019-01-01
124	DE	approved	2000	2019-01-07

MySQL Solution

```
</>Code
MySQL v Auto
1 # Write your MySQL query statement below
2 SELECT
3     DATE_FORMAT(trans_date, '%Y-%m') AS month,
4     country,
5     COUNT(*) AS trans_count,
6     SUM(CASE
7         WHEN state = 'approved' THEN 1
8         ELSE 0
9     END) AS approved_count,
10    SUM(amount) AS trans_total_amount,
11    SUM(CASE
12        WHEN state = 'approved' THEN amount
13        ELSE 0
14    END) AS approved_total_amount
15 FROM Transactions
16 GROUP BY DATE_FORMAT(trans_date, '%Y-%m'), country;
```

Testcase > Test Result

Output

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

View less

6. Last Person to Fit in the Bus

Table: Queue

Column Name	Type
person_id	int
person_name	varchar
weight	int
turn	int

person_id column contains unique values.

This table has the information about all people waiting for a bus.

The person_id and turn columns will contain all numbers from 1 to n, where n is the number of rows in the table.

turn determines the order of which the people will board the bus, where turn=1 denotes the first person to board and turn=n denotes the last person to board.

weight is the weight of the person in kilograms.

Problem Summary

There is a queue of people waiting to board a bus. However, the bus has a weight limit of 1000 kilograms, so there may be some people who cannot board.

Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

Note that only one person can board the bus at any given turn.

SQL Schema

Create table If Not Exists Queue (person_id int, person_name varchar(30), weight int, turn int)

Truncate table Queue

insert into Queue (person_id, person_name, weight, turn) values ('5', 'Alice', '250', '1')

insert into Queue (person_id, person_name, weight, turn) values ('4', 'Bob', '175', '5')

insert into Queue (person_id, person_name, weight, turn) values ('3', 'Alex', '350', '2')

insert into Queue (person_id, person_name, weight, turn) values ('6', 'John Cena', '400', '3')

insert into Queue (person_id, person_name, weight, turn) values ('1', 'Winston', '500', '6')

insert into Queue (person_id, person_name, weight, turn) values ('2', 'Marie', '200', '4')

Queue

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

MySQL Solution

```
</> Code
MySQL Auto
1 # Write your MySQL query statement below
2 WITH RunningTotal AS (
3     SELECT
4         person_name,
5         turn,
6         SUM(weight) OVER (ORDER BY turn) AS total_weight
7     FROM Queue
8 )
9
10 SELECT person_name
11 FROM RunningTotal
12 WHERE total_weight <= 1000
13 ORDER BY turn DESC
14 LIMIT 1;
```

Testcase Test Result

Output

person_name
John Cena

7. Restaurant Growth

Table: Customer

Column Name	Type
customer_id	int
name	varchar
visited_on	date
amount	int

In SQL, (customer_id, visited_on) is the primary key for this table.

This table contains data about customer transactions in a restaurant.

visited_on is the date on which the customer with ID (customer_id) has visited the restaurant.

amount is the total paid by a customer.

Problem Summary

You are the restaurant owner and you want to analyze a possible expansion (there will be at least one customer every day).

Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before). average_amount should be rounded to two decimal places.

Return the result table ordered by visited_on in ascending order.

SQL Schema

Create table If Not Exists Customer (customer_id int, name varchar(20), visited_on date, amount int)

Truncate table Customer

insert into Customer (customer_id, name, visited_on, amount) values ('1', 'Jhon', '2019-01-01', '100')

insert into Customer (customer_id, name, visited_on, amount) values ('2', 'Daniel', '2019-01-02', '110')

insert into Customer (customer_id, name, visited_on, amount) values ('3', 'Jade', '2019-01-03', '120')

insert into Customer (customer_id, name, visited_on, amount) values ('4', 'Khaled', '2019-01-04', '130')

insert into Customer (customer_id, name, visited_on, amount) values ('5', 'Winston', '2019-01-05', '110')

insert into Customer (customer_id, name, visited_on, amount) values ('6', 'Elvis', '2019-01-06', '140')

insert into Customer (customer_id, name, visited_on, amount) values ('7', 'Anna', '2019-01-07', '150')

insert into Customer (customer_id, name, visited_on, amount) values ('8', 'Maria', '2019-01-08', '80')

insert into Customer (customer_id, name, visited_on, amount) values ('9', 'Jaze', '2019-01-09', '110')

insert into Customer (customer_id, name, visited_on, amount) values ('1', 'Jhon', '2019-01-10', '130')

insert into Customer (customer_id, name, visited_on, amount) values ('3', 'Jade', '2019-01-10', '150')

Customer

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140
7	Anna	2019-01-07	150
8	Maria	2019-01-08	80
9	Jaze	2019-01-09	110
1	Jhon	2019-01-10	130
3	Jade	2019-01-10	150

MySQL Solution

 Code

MySQL 

```
1 # Write your MySQL query statement below
2 WITH daily AS (
3     SELECT
4         visited_on,
5         SUM(amount) AS daily_total
6     FROM Customer
7     GROUP BY visited_on
8 )
9
10 SELECT
11     d1.visited_on,
12     SUM(d2.daily_total) AS amount,
13     ROUND(SUM(d2.daily_total) / 7, 2) AS average_amount
14 FROM daily d1
15 JOIN daily d2
16     ON d2.visited_on BETWEEN DATE_SUB(d1.visited_on, INTERVAL 6 DAY)
17     AND d1.visited_on
18 GROUP BY d1.visited_on
19 HAVING COUNT(d2.visited_on) = 7
20 ORDER BY d1.visited_on;
```

 Testcase  Test Result

Output

visited_on	amount	average_amount
2019-01-07	860	122.86
2019-01-08	840	120
2019-01-09	840	120
2019-01-10	1000	142.86

8. Movie Rating

Table: Movies

Column Name	Type
movie_id	int
title	varchar

movie_id is the primary key (column with unique values) for this table.
title is the name of the movie.

Table: Users

Column Name	Type
user_id	int
name	varchar

user_id is the primary key (column with unique values) for this table.
The column 'name' has unique values.

Table: MovieRating

Column Name	Type
movie_id	int
user_id	int
rating	int
created_at	date

(movie_id, user_id) is the primary key (column with unique values) for this table.
This table contains the rating of a movie by a user in their review.
created_at is the users review date.

Problem Summary

Write a solution to:

Find the name of the user who has rated the greatest number of movies. In case of a tie, return the lexicographically smaller user name.

Find the movie name with the highest average rating in February 2020. In case of a tie, return the lexicographically smaller movie name.

SQL Schema

Create table If Not Exists Movies (movie_id int, title varchar(30))

Create table If Not Exists Users (user_id int, name varchar(30))

Create table If Not Exists MovieRating (movie_id int, user_id int, rating int, created_at date)

Truncate table Movies

insert into Movies (movie_id, title) values ('1', 'Avengers')

insert into Movies (movie_id, title) values ('2', 'Frozen 2')

insert into Movies (movie_id, title) values ('3', 'Joker')

Truncate table Users

insert into Users (user_id, name) values ('1', 'Daniel')

insert into Users (user_id, name) values ('2', 'Monica')

insert into Users (user_id, name) values ('3', 'Maria')

insert into Users (user_id, name) values ('4', 'James')

Truncate table MovieRating

insert into MovieRating (movie_id, user_id, rating, created_at) values ('1', '1', '3', '2020-01-12')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('1', '2', '4', '2020-02-11')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('1', '3', '2', '2020-02-12')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('1', '4', '1', '2020-01-01')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('2', '1', '5', '2020-02-17')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('2', '2', '2', '2020-02-01')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('2', '3', '2', '2020-03-01')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('3', '1', '3', '2020-02-22')

insert into MovieRating (movie_id, user_id, rating, created_at) values ('3', '2', '4', '2020-02-25')

Movies

movie_id	title
1	Avengers
2	Frozen 2
3	Joker

Users

user_id	name
1	Daniel
2	Monica
3	Maria
4	James

MovieRating

movie_id	user_id	rating	created_at
1	1	3	2020-01-12
1	2	4	2020-02-11
1	3	2	2020-02-12
1	4	1	2020-01-01
2	1	5	2020-02-17
2	2	2	2020-02-01
2	3	2	2020-03-01
3	1	3	2020-02-22
3	2	4	2020-02-25

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 (
3     -- Part 1: User who rated the most movies
4     SELECT u.name AS results
5     FROM MovieRating mr
6     JOIN Users u
7     ON mr.user_id = u.user_id
8     GROUP BY mr.user_id
9     ORDER BY COUNT(*) DESC, u.name ASC
10    LIMIT 1
11 )
12
13 UNION ALL
14
15 (
16     -- Part 2: Movie with highest average rating in Feb 2020
17     SELECT m.title AS results
18     FROM MovieRating mr
19     JOIN Movies m
20     ON mr.movie_id = m.movie_id
21     WHERE DATE_FORMAT(mr.created_at, '%Y-%m') = '2020-02'
22     GROUP BY mr.movie_id
23     ORDER BY AVG(mr.rating) DESC, m.title ASC
24     LIMIT 1
25 );
```

Testcase Test Result

Output

results

Daniel
Frozen 2

9. Customers Who Bought All Products

Table: Customer

Column Name	Type
customer_id	int
product_key	int

This table may contain duplicates rows.

customer_id is not NULL.

product_key is a foreign key (reference column) to Product table.

Table: Product

Column Name	Type
product_key	int

product_key is the primary key column for this table.

Problem Summary

Write a solution to report the customer ids from the Customer table that bought all the products in the Product table.

SQL Schema

Create table If Not Exists Customer (customer_id int, product_key int)

Create table Product (product_key int)

Truncate table Customer

insert into Customer (customer_id, product_key) values ('1', '5')

insert into Customer (customer_id, product_key) values ('2', '6')

insert into Customer (customer_id, product_key) values ('3', '5')

insert into Customer (customer_id, product_key) values ('3', '6')

insert into Customer (customer_id, product_key) values ('1', '6')

Truncate table Product

insert into Product (product_key) values ('5')

insert into Product (product_key) values ('6')

Customer

customer_id	product_key
1	5
2	6
3	5
3	6
1	6

Product

product_key
5
6

MySQL Solution

```
Code
MySQL Auto
1 # Write your MySQL query statement below
2 SELECT customer_id
3 FROM Customer
4 GROUP BY customer_id
5 HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(DISTINCT product_key) FROM Product);
```

```
Testcase Test Result
Output
| customer_id |
|-----|
| 1           |
| 3           |
```